ZefasBI & Analytics

Building a Fabric Data App

A concise field guide — from a semantic model to a live, AI-built, Fabric-hosted app.

Quick Guide · v2
Microsoft Fabric + Rayfin
+ Claude Code / Copilot

1 · The pipeline, at a glance

```
Excel in OneDrive  →  Dataflow Gen2  →  Lakehouse  →  Direct Lake semantic model  →  Fabric Data App
   (cloud-to-cloud, no gateway)        (Delta tables)     (relationships, DAX measures)   (React +
Rayfin, built by an AI agent)
```

A data app connects to a **semantic model** and queries it in DAX. So the model is a required link — build it (with relationships and measures) before scaffolding the app.

✓ **WHY CLOUD-TO-CLOUD** A file on your local drive can't refresh in the cloud without a gateway. OneDrive → Dataflow → Lakehouse removes the gateway entirely.

2 · Set up the environment

1. **Node.js (LTS)** — install, then **open a new terminal**. Verify `node -v`.
2. **VS Code** — your project home and your agent's workspace.
3. **AI agent** — Claude Code (`npm i -g @anthropic-ai/claude-code`) or GitHub Copilot.
4. **Azure CLI** — needed so the agent can query your model: `az login --allow-no-subscriptions --tenant <id>`.

⚠ **"not recognized" after install** — PATH only refreshes in **new** terminals. → **Close & reopen the terminal.** (The #1 stumble — applies to node, az, claude.)

⚠ **"No subscriptions found" on az login** — Fabric account, no Azure sub. → **Add `--allow-no-subscriptions`.**

Find your tenant ID: `az account show --query tenantId -o tsv` (or the `ctid=` value in any Fabric portal URL). **Workspace ID** is in the workspace URL: `/groups/<workspace-id>/`.

Out of Copilot credits? Switch to Claude Code — your project doesn't care which agent edits it.

GitHub Copilot's free tier has a monthly credit limit; new paid sign-ups can also be paused. When it stops mid-build, the project is **not** tied to Copilot — any agent can pick up in the same folder.

1. **Keep your work.** Click **Keep** on any staged "files changed" panel so the in-progress edits aren't lost.
2. **Install Claude Code.** `npm install -g @anthropic-ai/claude-code`
3. **Launch it** in the same project folder: `claude` — sign in with your Claude account; it reads the same `AGENTS.md`.
4. **Continue.** Tell it to read `AGENTS.md` and review the files Copilot last touched, then resume.

⚠ **Copilot "monthly credit limit reached"** — stops mid-edit. → **Hit Keep first, then switch agents — the build isn't lost.**

✓ **TIP** Claude Code uses your Claude plan's agent credits (separate from Copilot). It and Copilot both read the project's `AGENTS.md`, so the swap costs you nothing but a re-launch.

3 · Scaffold & provision

```
cd "your projects folder"
npm create @microsoft/rayfin@latest -- "MyApp" --template dataapp --workspace "Workspace_Name"
# then, inside the new folder:
npx rayfin login --tenant <tenant-id>
npx rayfin up   --workspace-id <workspace-id>
```

`rayfin up` creates the Fabric app item and writes the env vars — **required before the dev window will open.**

⚠ **403 "feature is not available"** — a tenant gate. → **A Fabric admin enables "Fabric App Items (preview)" and "Semantic Model Execute Queries REST API"; wait ~15 min to propagate, re-run.**

4 · Test vs. Deploy — the part everyone trips on

This is the single most confusing thing, so read it once carefully. **A data app needs two things at the same time: your code, and Fabric's login.** Neither alone is enough — and that's why there are three URLs that look similar but do very different things.

✕ Plain localhost:5173
Your code, **no Fabric login**. Always shows *"Can't open this app outside Fabric."* Not an error — just the wrong window. **Never build here.**

◖ ?experience=power-bi
The **deployed / production** app. Hosted in Fabric, frozen at your last `rayfin up`. **For sharing.** Does not show your live edits.

✓ …&devUri=localhost:5173
Your **live local code wrapped in Fabric's login**. Hot-reloads every edit. **This is your test/dev environment — build here.**

The key idea: the `devUri` parameter tells Fabric *"don't run your built-in code — load it live from my localhost instead."* That single parameter is the "wrap" that combines your code with Fabric's login. So:

- **localhost** = your code, no login → the error screen.
- **?experience=power-bi** = login, but old deployed code.
- **devUri** = your live code *inside* the login → the only window where you see work-in-progress.

The build loop (do this every session)

```
npm run dev          # start the local code server — keep it running
npm run test:fabric  # opens the devUri window — build & check here
```

Edit code (via the agent) → it hot-reloads into the devUri window. When you hit a **milestone**, run `npx rayfin up` to publish it to the production URL. **Test in dev; deploy at milestones** — you'll run `rayfin up` many times across a project, not once.

⚠ **"Can't open outside Fabric"** — you opened plain localhost. → **Don't fix that tab. Run `npm run test:fabric` and use the window it opens.**

⚠ **The test:fabric window never appears** — Playwright launched it **headless** (invisible). → **Set `headless: false` in `.playwright-config.json`, then re-run. (Also check taskbar / alt-tab / second monitor.)**

⚠ **"My change isn't showing"** — you're on the wrong/stale window. → **Confirm the URL has `devUri=localhost` (not `?experience=power-bi`); hard-refresh (Ctrl+Shift+R); keep exactly one window open.**

✓ **TEACHING TIP** Tell learners up front: "localhost shows an error on purpose — it needs Fabric's login. Build in the `test:fabric` window. If a change doesn't show, you're on the wrong window." This prevents 90% of confusion.

5 · Wire the data (the build)

1. **Connect & discover.** Point the agent at your model's URL, then have it run schema discovery — never let it guess table/column/measure names.
2. **Prove on one region.** Wire the KPI cards first; this builds the reusable query plumbing once, where you can verify numbers instantly.
3. **Roll out the rest.** Each region reuses the pattern: a `.dax` query → a factory with filters → wire into the component. Validate every DAX query against the model.
4. **Build in passes, not all at once.** Layout/look first, then data; verify each in the dev window before the next.

The live-data test: change a slicer and watch the numbers move. That's the difference between a picture and a working app.

> ⚠ **A column the agent needs isn't in the model** — a schema-sync gap. → **Re-run the dataflow → SQL endpoint Refresh → Edit tables / Reload (Direct Lake doesn't auto-detect new columns).**

> ⚠ **Text/layout collapses or overlaps** — the template's Tailwind theme omits some tokens. → **Use explicit bracket values like `leading-[1.5]` instead of named utilities.**

> ✓ **MATCH AN EXISTING DESIGN** Drop a reference HTML mock into the project (e.g. a `.sample/` folder) and tell the agent to port its rendering/styling and feed it live data. Faster and more faithful than describing the look in words.

6 · Quick troubleshooting reference

Symptom	Fix
"not recognized" (node/az/claude)	Close & reopen the terminal
"running scripts is disabled" (PowerShell)	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Agent: "az: command not found"	Run CLI via `powershell.exe -Command "..."` (az is on the Windows PATH; agent uses bash)
Source needs a gateway	Use the cloud SharePoint/OneDrive URL, not the local path
New column missing in model	SQL endpoint Refresh → Edit tables / Reload
Deploy: 403 "feature not available"	Enable the two tenant settings; wait ~15 min
"Can't open outside Fabric"	Use the `test:fabric` (devUri) window, not plain localhost
Change isn't showing	Wrong/stale window — check for `devUri`, hard-refresh, one window only
test:fabric window invisible	`headless: false` in `.playwright-config.json`

7 · Golden rules

- **Cloud-to-cloud beats gateways** · **new terminal after every install** · **route CLI through the shell that owns it** (PowerShell for az/rayfin).
- **Build in dev (devUri), publish at milestones (`rayfin up`).** If a change isn't showing, suspect the window before the code.
- **Never accept mock data** — make the agent discover real names and validate every query. **Prove the pipeline on one region first.**
- **Direct Lake: data auto-syncs, schema doesn't** — new columns need a refresh + Edit tables/Reload.